



БЛОЧНАЯ СТРУКТУРА

Введение

SQL

Язык запросов

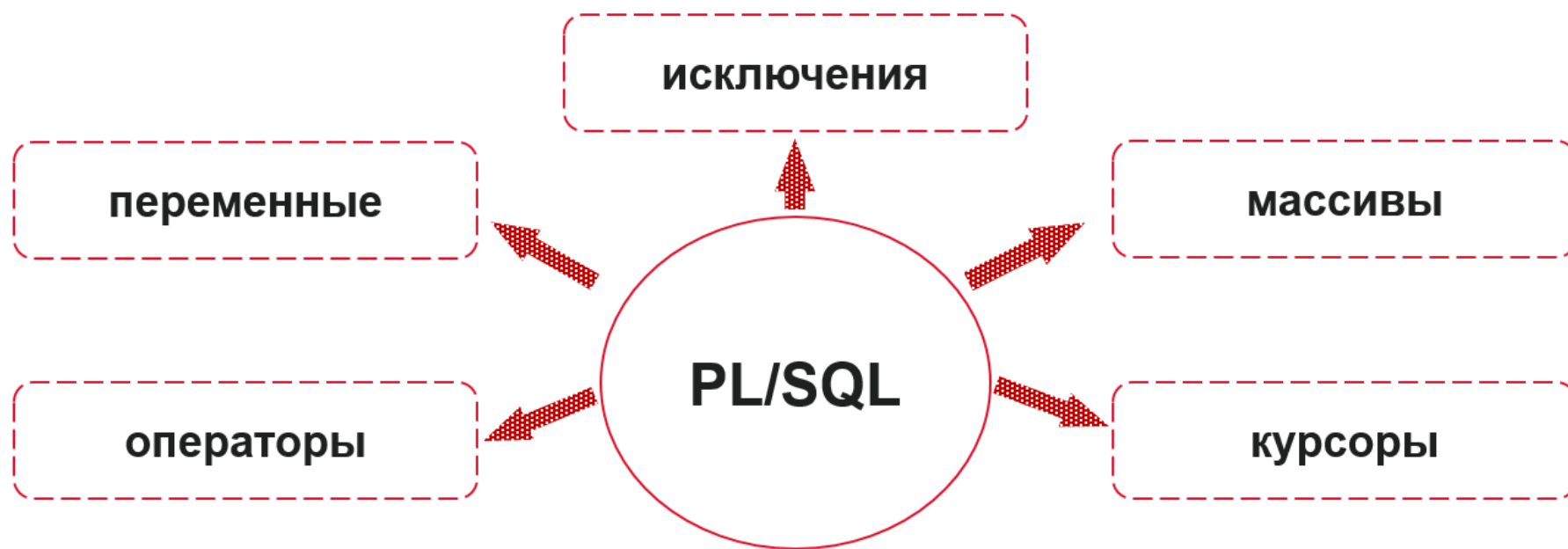


PL/SQL

**Процедурное
расширение
языка запросов**



Введение



...

Переменные

Переменная или, по-другому, идентификатор , ID (англ. data name, identifier — опознаватель) — уникальный признак объекта, позволяющий отличать его от других объектов, то есть идентифицировать.

Переменные должны обладать следующими свойствами:

- Иметь длину до 63 байтов (иначе будет усечено)
- Должны начинаться с буквы или подчеркивания (_)
- Могут включать в себя знаки доллара \$, подчеркивания (_)
- Не могут содержать никакие пробельные символы

Например: x, v1, v1_, v_Student, vID

Переменные

Вы можете задать кавычковое имя для переменной

Например: "My Var " , "user"

В таком случае можно использовать любые символы, включая пробелы, дефисы, специальные знаки, кириллицу и т.д. При каждом обращении к нему необходимо обязательно использовать эти кавычки и точный регистр.

Рекомендация: придерживайтесь некавыченных имён, начинающихся с буквы или подчёркивания, используйте только латиницу, цифры и `_`. Это делает код переносимым, читаемым и лишённым неожиданных проблем с регистром.

Переменные

Если два идентификатора отличаются только регистром одной или нескольких букв, то PL/SQL воспринимает их как один и тот же идентификатор. Например, следующие идентификаторы для PL/SQL идентичны:

lots_of_MONEY

=

LOTS_of_MONEY

=

Lots_of_Money

Переменные

Каждая переменная должна быть объявлена определенным типом данных.

Например:

```
vDescription    VARCHAR(2000) ;  
vSum1           integer ;  
vSum2           numeric(9,2) ;  
vResult         boolean ;  
vBirthDate      date ;
```

PL/pgSQL поддерживает все типы данных, доступные в PostgreSQL

Типы данных

Числовые целочисленные типы:

- Smallint (целые числа в диапазоне от -32,768 до 32,767)
- Integer (целые числа в диапазоне от -2,147,483,648 до 2,147,483,647)
- Bigint (целые числа в диапазоне от -9,223,372,036,854,775,808 до 9,223,372,036,854,775,807)

Типы данных

Числа фиксированной точности:

- Numeric – предназначен для хранения чисел с фиксированной точностью (например, финансовых данных), поддерживая до 131072 цифр до запятой и до 16383 после.
- Decimal – псевдоним типа Numeric.

Имеют два параметра – точность и масштаб.

Например, число 1234.55 имеет точность 6 цифр, масштаб – 2 цифры. Тип данных для такого числа будет задан как *numeric(6,2)*.

Типы данных

Числа с плавающей запятой:

- Real - точность в пределах 6 десятичных цифр. Имеет погрешность. Псевдоним – Float4.
- Double precision - точность в пределах 15 десятичных цифр. Более точный. Псевдоним – Float8.

Типы данных

Символьные (строковые) типы:

- Character varying (n) / varchar (n) – текстовый тип с указанием максимально допустимого количества символов.
- Character (n) / char (n) – текстовый тип с точным указанием количества символов.
- Text – безлимитный текстовый тип (до 1 Гб на одно поле).

Varchar (~~n~~) = Text
Char (~~n~~) = Char(1)

Типы данных

Типы Дата/время:

- Date – хранит дату без времени.
- Time – хранит время суток.
- Time with time zone / Timetz – хранит время суток и часовой пояс.
- Timestamp – хранит дату и время суток.
- Timestamp with time zone / Timestamptz – хранит дату, время суток и часовой пояс.

Типы данных

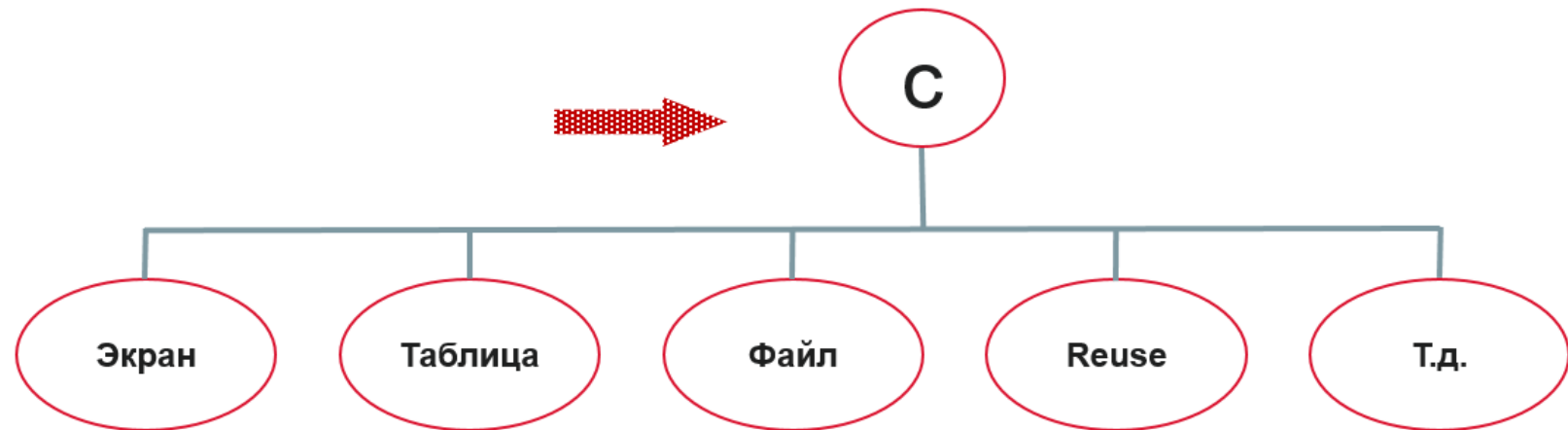
Логический тип. Boolean. Принимает 3 состояния:

- Истина. Может принимать значения: TRUE, 't', 'true', 'y', 'yes', 'on', '1'.
- Ложь. Может принимать значения: FALSE, 'f', 'false', 'n', 'no', 'off', '0'.
- Неопределенное состояние. Принимает единственное значение – NULL.

Операторы

Оператор – это элемент языка, который задает полное описание некоторого действия, которое необходимо выполнить. Каждый оператор представляет собой законченную фразу языка программирования и определяет некоторый этап обработки данных, например:

a := 10;
b := 5;
c := a – b;



Операторы

— Операторы:

- Арифметические операторы: + - * / ** и т.д.
- Операторы отношения (используются в логических выражениях): < > = != и т.д.
- Выражение и списки: := () , ' || и т.д.
- Комментарии:

--	Комментарий в одной строке
/*	Начало многострочного комментария
*/	Конец многострочного комментария

Операторы

RAISE

DEBUG

Для отладки.

Сообщение отправляется **только в лог-файл сервера** (обычно не видно клиенту).

LOG

Для логирования.

Сообщение отправляется **только в лог-файл сервера**, но с более высоким приоритетом

INFO

Для общей информации.

Сообщается **клиенту** и записывается в лог

NOTICE

Для уведомлений.

Сообщается **клиенту** и записывается в лог. Используется по умолчанию для информационных сообщений.

WARNING

Для предупреждений.

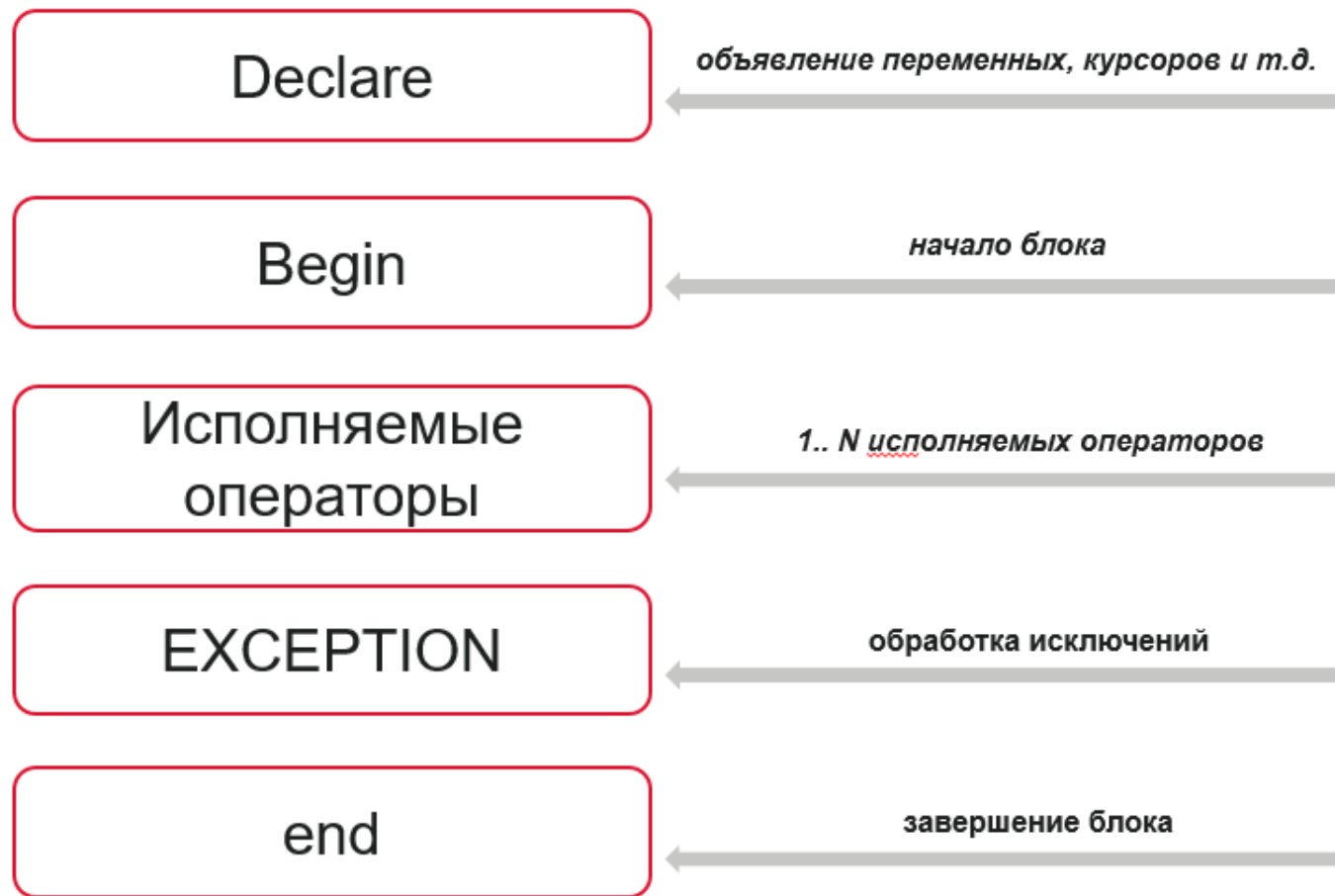
Сообщается **клиенту** и записывается в лог с более высоким приоритетом.

EXCEPTION

Для вызова ошибки.

(По умолчанию). Вызывает ошибку, которая прерывает текущую транзакцию (если не перехвачена в блоке **EXCEPTION**).

Блочная структура



```
DO $$  
Declare  
    vDate date;  
Begin  
    vDate := current_date;  
    RAISE NOTICE 'vDate = %',  
        vDate;  
  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'error';  
  
end;  
$$
```

Data Output Messages Notifications

NOTICE: vDate = 2026-08-27

Блочная структура

```
DO $$  
Declare  
    vDate    date;  
Begin  
    vDate := current_date;  
    insert into emp (EMPNO, HIREDATE) values (1, vDate);  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'error';  
end;  
$$
```

Блочная структура

```
DO $$  
Declare  
    vAmount1 int;  
    vAmount2 int;  
    vAmount3 int;  
    vResult  int;  
Begin  
    vAmount1 := 10;  
    vAmount2 := 20;  
    vAmount3 := 30;  
    vResult  := vAmount1 + vAmount2 + vAmount3;  
    RAISE NOTICE  
        'vResult = %', vResult;  
end;  
$$
```

Data Output	Messages
-------------	----------

NOTICE:	vResult = 60
---------	--------------

Блочная структура

```
DO $$  
Declare  
    vAmount1 int;  
    vAmount2 int;  
    vAmount3 int;  
    vResult  int;  
Begin  
    vAmount1 := 10;  
    vAmount2 := 20;  
    vResult  := vAmount1 + vAmount2 + vAmount3;  
    RAISE NOTICE  
        'vResult = %', vResult;  
end;  
$$
```

Data Output	<u>Messages</u>	No
NOTICE: vResult = <NULL>		

Блочная структура

```
DO $$  
Declare  
    vAmount1 int;  
    vAmount2 int;  
    vAmount3 int;  
    vResult  int;  
Begin  
    vAmount1 := 10;  
    vAmount2 := 20;  
    vResult  := coalesce(vAmount1, 0) +  
                coalesce(vAmount2, 0) +  
                coalesce(vAmount3, 0);  
  
    RAISE NOTICE  
        'vResult = %', vResult;  
end;  
$$
```

Data Output	Messages
NOTICE: vResult = 30	

Блочная структура

```
DO $$
```

```
Declare
```

```
  vDate date;
```

```
  vCount int;
```

```
  vText1 varchar(100);
```

```
  vText2 varchar(100);
```

```
  vText3 varchar(100);
```

```
Begin
```

```
  vDate := current_date-1;
```

```
  vCount := 5;
```

```
  vText1 := 'Вчера';
```

```
  vText2 := 'я выполнил/а';
```

```
  vText3 := 'задач/и.';
```

```
  RAISE NOTICE
```

```
    'Результат = % %, % % %', vText1, vDate, vText2, vCount, vText3;
```

```
end;
```

```
$$
```

```
NOTICE:  Результат = Вчера 2026-08-29, я выполнил/а 5 задач/и.
```

Блочная структура

```
DO $$  
Declare  
    vDate date := current_date-1;  
    vCount int := 5;  
    vText1 varchar(100) := 'Вчера';  
    vText2 varchar(100) := 'я выполнил/а';  
    vText3 varchar(100) := 'задач/и.';  
Begin  
    RAISE NOTICE  
        'Результат = % %, % % %', vText1, vDate, vText2, vCount, vText3;  
end;  
$$  
        NOTICE:  Результат = Вчера 2026-08-29, я выполнил/а 5 задач/и.
```

Блочная структура

```
DO $$  
Declare  
    vDate date := current_date-1;  
    vCount int := 5;  
    vText1 varchar(100) := 'Вчера';  
    vText2 varchar(100) := 'я выполнил/а';  
    vText3 varchar(100) := 'задач/и.';  
Begin  
    vCount := 6;  
    RAISE NOTICE  
        'Результат = % %, % % %', vText1, vDate, vText2, vCount, vText3;  
end;  
$$
```

NOTICE: Результат = Вчера 2026-08-29, я выполнил/а 6 задач/и.

Блочная структура

Constant

Константа – постоянное, неизменяемое значение.

Шаблон:

[имя]_constant_[тип данных]_[оператор присваивания значений]_[значение];

Пример:

```
cCount constant int := 5;
```

Блочная структура

```
DO $$  
Declare  
  vDate constant date := current_date-1;  
  vCount constant int := 5;  
  vText1 constant varchar(100) := 'Вчера';  
  vText2 constant varchar(100) := 'я выполнил/а';  
  vText3 constant varchar(100) := 'задач/и.';  
Begin  
  vCount := 6;  
  RAISE NOTICE  
    'Результат = % %, % % %', vText1, vDate, vText2, vCount, vText3;  
end;  
$$
```

Блочная структура

Data Output Messages Notifications

```
ERROR:  variable "vcount" is declared CONSTANT  
LINE 9:  vCount := 6;  
          ^
```

Блочная структура

Общепринятые правила именования переменных:

- **c** – константа
- **v** – локальная переменная
- **cVal, c_Val, cVAL, c_VAL, CVAL, C_VAL**
- **vVal, v_Val, vVAL, v_VAL, VVAL, V_VAL**

Блочная структура

DO \$\$

Declare

```
cSep constant varchar(1) := '|';  
vLine  varchar(40) := '-----';  
vText1 varchar(100) := '1 место';  
vText2 varchar(100) := '2 место';  
vText3 varchar(100) := '3 место';  
vResult varchar(304) ;
```

Begin

```
vResult := cSep || vText1 || cSep || vText2 ||  
           cSep || vText3 || cSep;
```

```
RAISE NOTICE '%', vLine;
```

```
RAISE NOTICE '%', vResult;
```

```
RAISE NOTICE '%', vLine;
```

end;

\$\$

```
NOTICE: -----  
NOTICE: |1 место|2 место|3 место|  
NOTICE: -----
```

Блочная структура

```
DO $$
```

```
Declare
```

```
  vLine    varchar(40) := '-----';  
  vText1    varchar(100) := '1 место';  
  vText2    varchar(100) := '2 место';  
  vText3    varchar(100) := '3 место';  
  vResult   varchar(304) ;
```

```
Begin
```

```
  vResult := '|' || vText1 || '|' || vText2 ||  
             '?' || vText3 || '|';
```

```
  RAISE NOTICE '%', vLine;
```

```
  RAISE NOTICE '%', vResult;
```

```
  RAISE NOTICE '%', vLine;
```

```
end;
```

```
$$
```

```
NOTICE: -----  
NOTICE: |1 место|2 место?3 место|  
NOTICE: -----
```



**Обогащая традиции
инновациями**

Спасибо!
У вас остались вопросы?



* изображение для этого слайда создано нейросетью Kandisky 2.2 по запросу «Изображение в серых тонах, которое олицетворяет синергию традиций и инноваций в финансовой сфере»